



Programa de la asignatura Laboratorio 2 de Computación

Universidad Nacional de San Martín

Escuela de Ciencia y Tecnología

Carrera: Tecnicatura universitaria en programación informática

Carga horaria: 8 horas semanales

Régimen: Presencial

Fundamentación

La asignatura *Laboratorio 2 de Computación* tiene como objetivo profundizar los conocimientos de programación adquiridos previamente. Se trabaja con tres lenguajes de programación: C, Python y C++, abordando desde programación estructurada hasta programación orientada a objetos.

Se busca que el estudiante comprenda el uso eficiente de estructuras de datos, administración de memoria, manejo de archivos y modularidad del código, integrando progresivamente conceptos más complejos a través de prácticas y proyectos. Esta materia proporciona las herramientas fundamentales para desarrollarse en áreas como sistemas embebidos, desarrollo de software, inteligencia artificial y simulación numérica.

Objetivos Generales

- Consolidar habilidades de programación utilizando C, Python y C++.
- Comprender el uso de punteros, estructuras, memoria dinámica y archivos en C.
- Desarrollar algoritmos aplicando estructuras de datos dinámicas en Python.
- Incorporar el paradigma orientado a objetos en C++ y Python.
- Aplicar los lenguajes estudiados a problemas concretos mediante prácticas de laboratorio.

Contenidos

1 Unidad 1: Lenguaje C

Objetivo

Dominar el lenguaje C, con énfasis en estructuras fundamentales, punteros, manejo de memoria y archivos.

Contenidos

1. **Variables, ciclos y control de flujo**

Tipos de datos primitivos, operadores aritméticos y lógicos, estructuras condicionales `if`, `switch`. Bucles `for`, `while`, `do-while`.

2. **Punteros**

Introducción al concepto de dirección de memoria. Punteros simples y dobles, aritmética de punteros, punteros y funciones.

3. **Funciones**

Declaración, definición e invocación de funciones. Paso por valor y por referencia. Alcance y duración de variables.

4. **Vectores**

Arreglos unidimensionales. Recorrido, inicialización, operaciones básicas. Vectores como parámetros de funciones.

5. **Vectores con cadenas de caracteres**

Manejo de strings como arrays de `char`. Funciones de la librería `string.h`. Comparación, copia, concatenación.

6. **Estructuras**

Declaración y uso de `struct`. Arreglos de estructuras. Estructuras anidadas. Estructuras como parámetros.

7. **Matrices**

Declaración e inicialización. Recorridos anidados. Paso como parámetro. Aplicaciones matemáticas.

8. **Memoria dinámica**

Uso de `malloc`, `calloc`, `realloc`, `free`. Aplicaciones con vectores dinámicos y estructuras enlazadas.

9. **Archivos de texto**

Funciones de manejo de archivos: `fopen`, `fclose`, `fprintf`, `fscanf`, `fgets`. Modos de apertura y recorrido.

10. Archivos binarios

Lectura y escritura con `fread` y `fwrite`. Comparación con archivos de texto. Serialización de estructuras.

11. Librerías

Modularización. Separación de código en headers y archivos fuente. Creación y uso de librerías propias.

2 Unidad 2: Lenguaje Python

Objetivo

Introducir al lenguaje Python y sus estructuras dinámicas. Abordar su paradigma multiparadigma con enfoque en objetos y manejo de datos.

Contenidos

1. **Introducción a Python**
Historia, características, instalación. Primeros scripts.
2. **Variables, listas y condicionales**
Tipos dinámicos. Listas y operaciones. Condicionales `if`, `elif`, `else`.
3. **Ciclos y range**
Iteración con `for` y `while`. Uso de `range`, comprensión de listas.
4. **Conjuntos y objetos**
Operaciones con `set`. Introducción a objetos, atributos y métodos.
5. **Diccionarios, tuplas y funciones**
Estructuras clave-valor. Tuplas inmutables. Declaración y retorno de funciones. Parámetros opcionales y por nombre.
6. **Generadores**
Uso de `yield`, funciones generadoras, eficiencia en memoria.
7. **Numpy y Matplotlib**
Introducción a `numpy` para arrays multidimensionales. Visualización básica con `matplotlib`.
8. **Decoradores**
Funciones como objetos. Funciones dentro de funciones. Uso de `@decorator`.
9. **Clases**
Definición de clases. Métodos de instancia y clase. Métodos especiales (`__init__`, `__str__`, etc).
10. **Pilares de la Programación Orientada a Objetos**
Encapsulamiento, herencia, polimorfismo, abstracción. Ejemplos aplicados.
11. **Archivos y excepciones**
Lectura y escritura de archivos con `open`, uso de `with`. Manejo de errores con `try/except/finally`.
12. **APIs con FastAPI**
Introducción a servicios web. Instalación de FastAPI. Creación de endpoints, manejo de rutas, parámetros, respuestas y documentación automática. Conexión con bases de datos (introducción).

3 Unidad 3: Lenguaje C++

Objetivo

Introducir los conceptos fundamentales del lenguaje C++ y su enfoque en la programación orientada a objetos.

Contenidos

1. **Diferencias entre C y C++**

Sintaxis, tipos de datos, entrada/salida con `cin/cout`. Manipulación de strings. Manejo de memoria con `new/delete`.

2. **Programación Orientada a Objetos con C++**

Clases, objetos, encapsulamiento. Constructores y destructores. Herencia. Sobrecarga de funciones. Polimorfismo y clases virtuales.

4 Unidad 4: Manejo de Repositorios con Git y GitHub

Objetivo

Incorporar buenas prácticas de control de versiones utilizando herramientas profesionales. Trabajar colaborativamente en proyectos compartidos.

Contenidos

1. **Conceptos básicos de control de versiones**
¿Qué es Git? Ventajas del versionado. Flujo de trabajo básico.
2. **Comandos esenciales**
`git init`, `add`, `commit`, `status`, `log`, `diff`, `branch`, `merge`.
3. **Trabajo con GitHub**
Creación de repositorios remotos. Clonado, `push`, `pull`, forks y pull requests.
4. **Buenas prácticas y trabajo colaborativo**
Readme, licencias, `.gitignore`, uso de issues y ramas. Integración con proyectos en C, Python y C++.

Metodología de Trabajo

- Clases teóricas breves seguidas de resolución de problemas prácticos en computadora.
- Prácticas de laboratorio supervisadas.
- Trabajo en grupos para fomentar la colaboración.
- Uso de plataformas de control de versiones y desarrollo colaborativo (opcional).

Evaluación

- **Parcial 1 (Lenguaje C): 25 %**
- **Parcial 2 (Python): 25 %**
- **Trabajo Final Integrador (C++): 25 %**
- **Coloquio (Contenidos de la materia): 25 %**

Es requisito aprobar cada instancia con una nota mínima de 4/10. Para promocionar deberá aprobar con nota mayor o igual a 7 todas las instancias de evaluación. El trabajo final puede incluir una presentación o defensa del código desarrollado.

Bibliografía Sugerida

- Kernighan, B. y Ritchie, D. – *El lenguaje de programación C*.
- Zelle, J. – *Python Programming: An Introduction to Computer Science*.
- Lajoie, J. y Liberty, J. – *Aprenda C++ en un fin de semana*.
- McKinney, W. – *Python for Data Analysis*.
- Recursos en línea: docs.python.org, cplusplus.com, [geeksforgeeks.org](https://www.geeksforgeeks.org).